

Adwene Pa

Team Roles & Responsibilities

Hackathon Build — 5 Team Members

Team Overview

	Role	Primary Responsibilities
Member 1	Project Lead & AI Engineer	System prompts, Claude API, crisis detection, integration, demo
Member 2	AI / NLP Engineer	Khaya ASR/TTS integration, voice pipeline, audio processing
Member 3	Mobile Developer (Frontend)	Conversation screen, crisis UI, toolkit, navigation
Member 4	Mobile Developer (Backend)	SQLite database, state management, data layer, services
Member 5	UX / Content & Presenter	UI design, onboarding, Twi content, coping audio, presentation

Contents

1. How to Use This Document
2. Member 1 — Project Lead & AI Engineer
3. Member 2 — AI / NLP Engineer
4. Member 3 — Mobile Developer (Frontend)
5. Member 4 — Mobile Developer (Backend)
6. Member 5 — UX / Content & Presenter
7. Shared Responsibilities & Handoff Points
8. Hour-by-Hour Build Schedule

9. Integration Checkpoints

10. Git Workflow & Branch Strategy

11. Environment Setup (Everyone)

12. Emergency Protocols

1. How to Use This Document

Each team member has a dedicated section that covers: their role summary, every file they own, exactly what they build, their hour-by-hour schedule, their dependencies on other members, and a checklist for completion. If a file is listed in your section, you are responsible for it. If it's not, don't touch it without coordinating with the owner.

Golden Rule: If you finish early, help the person who depends on your output. Don't start building features that aren't in scope — scope creep is how hackathons fail.

The build order is designed so that the most critical and risky pieces are built first. If the hackathon ends early, you'll have a working conversation with crisis detection but a rough UI. That scores higher than a polished UI with a broken AI backend.

Member 1 — Project Lead & AI Engineer

You own the brain of the app. The system prompts ARE the product.

Role Summary

You are responsible for the core AI logic: the system prompts that define how Claude behaves, the Claude API integration, the crisis detection embedded in structured output, and overall project coordination. You are the final decision-maker on scope questions. You also lead the hackathon demo and presentation.

Files You Own

File / Component	What You Build
src/services/llm/claudeClient.ts	Anthropic API client. Streaming responses. API key from env. Retry on 529 (overloaded)
src/services/llm/conversationService.ts	Manages conversation sessions: builds messages array (system prompt + last 10 turns)
src/services/llm/systemPrompts/base.ts	THE most important file. Master system prompt: identity, tone, clinical boundaries (never
src/services/llm/systemPrompts/twi.ts	Twi-specific addendum: idioms for distress, expected input patterns, response style, exa
src/utls/riskScoring.ts	Maps risk_level from Claude response to numeric score. Thresholds: 0-2=none, 3-5=low
docs/CLINICAL_GUIDELINES.md	The four non-negotiable rules, safeguard statement (English + Twi), spiritual framing res
docs/CRISIS_PROTOCOL.md	Step-by-step crisis detection and escalation system, risk level definitions, contact inform
.env.example	Template with ANTHROPIC_API_KEY and KHAYA_API_KEY variables.

Detailed Task Breakdown

Task 1: Write base.ts System Prompt

This is your first and most important task. Start here before anything else.

- Define Claude's identity as Adwene Pa — warm, unhurried, never clinical
- Write the structured output schema that every response must follow:

```
{ "response": "string", "risk_level": "none|low|moderate|high",  
  "risk_indicators": [] }
```

- Define the four clinical boundaries as hard rules in the prompt
- Write crisis detection rules: what constitutes high/moderate/low/none risk

- Define conversation modes: active_listening, guided_support, crisis
- Write the spiritual framing response pattern (acknowledge → validate both → bridge → offer)
- Specify response length constraints: max 2-3 sentences, short sentences for voice delivery
- Test in Anthropic API console or Postman with 20+ test inputs before integrating

Task 2: Write twi.ts Addendum

- Research and document Twi idioms for distress:
 - "me tirim gyegye me" — my head is disturbing me
 - "me kra mu nni h■" — my soul is not there (feeling empty)
 - "mey■ basaa" — I am scattered/broken
 - "me bo afono" — my stomach is full (overwhelmed)
- Specify Twi response style: use proverbs naturally, don't force them
- Define Twi suicide-related indirect expressions the prompt should detect
- List Twi phrases to avoid (overly formal, borrowed clinical English terms)
- Test with Twi inputs — have a Twi-speaking teammate verify naturalness

Task 3: Build claudeClient.ts

- Initialize Anthropic API client with streaming enabled
- Handle API key injection from environment variables
- Implement retry logic for 529 (overloaded) with exponential backoff
- Set default parameters: model, max_tokens=300, temperature=0.7, stream=true

Task 4: Build conversationService.ts

- Accept user message and current language code
- Load last 10 turns from Member 4's conversationRepository
- Build messages array: system prompt (base + twi if language=tw) + conversation history
- Call Claude via claudeClient with streaming
- Parse the structured JSON response to extract: response text, risk_level, risk_indicators
- Return parsed response to the calling hook (Member 3's useConversation)
- Save both user message and assistant response to SQLite via Member 4's repository

Task 5: Build riskScoring.ts

- Accept risk_level string from Claude response
- Map to numeric score: none=0, low=3, moderate=7, high=10
- Maintain rolling window of last 5 scores
- Return current risk assessment based on window (avoid single-message false positives)
- Export a shouldTriggerCrisis() function that Member 3's crisis modal uses

Task 6: Lead Demo & Presentation

- Own the 90-second demo flow — rehearse until smooth
- Prepare the three mandatory answers (who, what could go wrong, how does it empower)

- Coordinate with Member 5 on presentation slides/talking points
- Handle judge Q&A — you are the person who can speak to the AI and ethics depth

Dependencies

You Need From	What	When
Member 4	conversationRepository API (createSession, addMessage, getSessionMessages)	Before you write conversationService.ts
Member 2	Working Khaya ASR to test Twi inputs end-to-end	After your system prompts are written
Member 5	Twi idiom validation — have them verify your twi.ts phrases	During twi.ts development

Completion Checklist

- base.ts system prompt written and tested with 20+ inputs in API console
- twi.ts addendum written and validated by a Twi speaker
- Structured output consistently returns valid JSON with risk_level
- claudeClient.ts connects to API, streams responses, retries on 529
- conversationService.ts builds messages, calls Claude, parses response, saves to DB
- riskScoring.ts correctly maps risk levels and maintains rolling window
- Crisis detection triggers at correct thresholds (tested with crisis-language inputs)
- CLINICAL_GUIDELINES.md and CRISIS_PROTOCOL.md written
- Demo flow rehearsed at least 3 times end-to-end
- Can answer all three mandatory hackathon questions confidently

Member 2 — AI / NLP Engineer

You own the voice. If the Twi voice loop works, we win.

Role Summary

You are responsible for the entire voice pipeline: recording user audio, sending it to Khaya ASR for Twi transcription, receiving text back, and converting Claude's text responses into spoken Twi via Khaya TTS. You also handle audio processing, format conversion, and the fallback strategy if Khaya is unavailable. The "wow" moment of the demo — speaking Twi and hearing Twi back — depends on you.

Files You Own

File / Component	What You Build
src/services/khaya/khayaClient.ts	Axios/fetch instance with Khaya base URL (https://translation.ghananlp.org/v1) and API key
src/services/khaya/asr.ts	Accepts audio file path (recorded .m4a or .wav), sends to Khaya ASR endpoint (POST /asr), returns transcription text
src/services/khaya/tts.ts	Accepts text and language code, calls Khaya TTS endpoint (POST /tts), returns audio data (base64 or file path)
src/services/audio/recorder.ts	Wraps expo-av Audio.Recording. Start, stop, pause, resume. Returns file URI after stop
src/services/audio/player.ts	Wraps expo-av Audio.Sound. Loads audio from URI (local file or TTS response). Play, pause, stop
src/services/audio/permissions.ts	Requests and checks microphone permission via expo-av. Returns status enum. Used by recorder
src/hooks/useVoiceInput.ts	Full voice input flow: permission check → start recording → stop recording → convert to text (via ASR)
src/hooks/useAudioPlayback.ts	Manages playback of a single audio file (coping tool or TTS response). Exposes: isPlaying, play, stop
src/utils/audioUtils.ts	Helpers: convert .m4a to .wav for Khaya ASR compatibility (if needed). Calculate audio duration

Detailed Task Breakdown

Task 1: Set Up Khaya API Access

- Apply for Khaya API key at ghananlp.org (do this FIRST — may take time)
- Test all endpoints in Postman before writing any code:
 - POST /asr — send a test .wav file of Twi speech
 - POST /tts — send Twi text, verify audio quality

- Document response formats, error codes, and rate limits
- Determine audio format requirements (sample rate, encoding, file type)

Task 2: Build `khayaClient.ts`

- Create Axios/fetch instance with base URL and API key header (Ocp-Apim-Subscription-Key)
- Implement 429 rate limiting handler with exponential backoff (1s, 2s, 4s, max 3 retries)
- Set request timeout (10s for ASR, 15s for TTS — voice needs to feel responsive)
- Log errors clearly for debugging during the hackathon

Task 3: Build `asr.ts`

- Accept file URI from `recorder.ts`
- Convert to FormData with correct content type
- Send to POST `/asr` with Twi language header
- Parse response and return transcribed text string
- On failure: return empty string and set error flag (don't crash — let text input take over)
- Log transcription timing for demo debugging

Task 4: Build `tts.ts`

- Accept text string and language code ('tw' or 'en')
- For Twi: call Khaya TTS endpoint, receive audio/mpeg stream
- For English: use expo-speech as fallback (free, no API call needed)
- Convert received audio to playable format for expo-av
- Implement simple cache: Map<string, audioURI> to avoid re-generating identical phrases
- Start TTS on first sentence boundary (coordinate with Member 3's StreamingText component)

Task 5: Build `recorder.ts` and `player.ts`

- `recorder.ts`: configure expo-av Recording with settings compatible with Khaya ASR
 - Sample rate: check Khaya docs (likely 16kHz mono)
 - Format: .wav or .m4a (convert if Khaya requires specific format)
 - Expose: `startRecording()`, `stopRecording()` → returns file URI
- `player.ts`: load audio from local file URI or remote URL
 - Support both coping tool audio (bundled .mp3) and TTS audio (runtime generated)
 - Expose: `play()`, `pause()`, `seek()`, `stop()`, `duration`, `position`
 - Unload sound objects when done to prevent memory leaks

Task 6: Build `useVoiceInput.ts` Hook

- This is the hook that Member 3's VoiceButton and VoiceInputOverlay call
- Flow: check mic permission → start recorder → wait for stop → send to ASR → return text
- Expose state: { `isRecording`, `isTranscribing`, `transcript`, `error` }
- Expose actions: { `startRecording()`, `stopRecording()`, `clearTranscript()` }
- Handle the full error chain: permission denied, recording failure, ASR failure, timeout

Task 7: Pre-Generate Coping Tool Audio

- Coordinate with Member 5 who writes the scripts
- Generate Twi audio via Khaya TTS from the translated scripts
- Generate English audio via ElevenLabs free tier or record with team member
- Edit in Audacity: trim silence, normalise volume, export at 128kbps mp3
- Deliver 4 final files: grounding-54321.mp3 and breathing-478.mp3 in tw/ and en/

Dependencies

You Need From	What	When
Khaya API team	API key approval and working endpoints	ASAP — apply immediately
Member 5	Coping tool scripts (English + Twi) for audio generation	Before Task 7
Member 3	VoiceButton component that calls your useVoiceInput hook	For integration testing
Member 1	Claude responses (text) to feed into your TTS pipeline	For end-to-end voice loop testing

Completion Checklist

- Khaya API key obtained and tested in Postman
- khayaClient.ts connects to Khaya, handles rate limiting
- asr.ts transcribes Twi audio to text successfully
- tts.ts converts Twi text to playable audio
- English fallback works via expo-speech
- recorder.ts records audio in Khaya-compatible format
- player.ts plays both bundled mp3 and runtime TTS audio
- useVoiceInput hook works end-to-end: record → transcribe → return text
- useAudioPlayback hook plays/pauses/stops correctly
- 4 coping tool audio files generated and placed in assets/audio/coping/
- Full voice loop tested: Twi speech → ASR → Claude → TTS → Twi audio playback
- Fallback strategy works: when Khaya fails, text input seamlessly takes over

Member 3 — Mobile Developer (Frontend)

You own the screens. Every pixel the user sees is your responsibility.

Role Summary

You build all the screens and UI components that the user interacts with. Your primary focus is the conversation screen (the core experience), the crisis UI (the most sensitive screen), and the toolkit. You consume hooks and services built by Members 1, 2, and 4 — you don't write API calls or database queries directly.

Files You Own

Screens

File / Component	What You Build
src/app/(tabs)/_layout.tsx	Tab bar with 3 tabs: Home, Toolkit, Profile. Tab labels in user's chosen language via Member 1.
src/app/(tabs)/home.tsx	Greeting by nickname in chosen language. Today's mood (if checked in). Large "Kasa m" button.
src/app/(tabs)/toolkit.tsx	Grid of 2 CopingCard components: Grounding (5-4-3-2-1) and Breathing (4-7-8). Name i
src/app/(tabs)/profile.tsx	Nickname display, language toggle (Twi/English), check-in streak, MoodChart (7-day), A
src/app/conversation/[sessionId].tsx	THE main screen. Voice button (primary, large), keyboard toggle (secondary). Transcrip
src/app/checkin/index.tsx	Emotion wheel (EmotionWheelPicker) + one voice/text follow-up question. Accessible fro
src/app/crisis/index.tsx	Full-screen crisis pathway. Not in navigation — only triggered by crisis detection. Warm
src/app/toolkit/[toolId].tsx	Tool detail: full instructions for selected coping tool in chosen language. Step-by-step dis
src/app/toolkit/player.tsx	Audio player screen for coping tool. Play/pause, elapsed/remaining time. Uses Member

Components

File / Component	What You Build
components/ui/VoiceButton.tsx	Large circular mic button. States: idle (pulse animation), recording (red), processing (spi
components/ui/EmotionWheel.tsx	Circular grid of 10 emotion icons with labels in chosen language. Tap to select; selected
components/ui/MessageBubble.tsx	Chat message bubble. User (right, muted green) and assistant (left, cream). Text + optio

components/ui/ AudioPlayer.tsx	Inline audio player strip: play/pause, time display. Used in toolkit screens.
components/ui/ LanguagePill.tsx	Small pill showing current language. Tappable to toggle Twi/English.
components/ui/ MoodChart.tsx	7-day line chart using react-native-svg. Gets data from Member 4's useMoodData hook.
components/ui/ CopingCard.tsx	Card for toolkit grid. Icon, name in chosen language, description, "Use now" CTA.
components/ui/ CrisisBar.tsx	Non-intrusive banner on mild risk signals. "If you're struggling, help is available." Tap to
components/ui/ LoadingWave.tsx	Three-dot wave animation while Claude generates a response.
components/ui/ SafetyCheckModal.tsx	Full-screen modal at crisis level. Not dismissible by back button. Acknowledgement text
components/conversation/ ChatInterface.tsx	Full conversation inner component. Manages scroll-to-bottom, keyboard avoidance, voice
components/conversation/ VoiceInputOverlay.tsx	Translucent overlay during recording. Shows recording indicator, elapsed time, "tap to st
components/conversation/ TranscriptDisplay.tsx	Scrollable list of MessageBubble components. Initial load from SQLite, streaming new m
components/conversation/ StreamingText.tsx	Renders Claude response token by token. Triggers Member 2's TTS after each sentence
components/conversation/ CrisisModal.tsx	Mid-conversation crisis modal. Transitions gracefully. Different from SafetyCheckModal
components/checkin/ EmotionWheelPicker.tsx	Full-screen check-in version of EmotionWheel. Larger, animated selection.
components/checkin/ MoodFollowUp.tsx	After emotion selection: voice or text prompt. Short, skippable.
components/checkin/ CheckinSummary.tsx	Post check-in summary. Emotion selected, validating one-liner, "Talk about it" button.
components/shared/ SafeAreaWrapper.tsx	Wraps all screens for safe area insets across iOS/Android.
components/shared/ LanguageText.tsx	Text component using i18n JSON lookup. Handles missing keys gracefully.
components/shared/ ErrorBoundary.tsx	React error boundary. Catches crashes, shows friendly error screen.

Detailed Task Breakdown

Task 1: Conversation Screen (Priority 1)

This is the core experience. Build it first, test it extensively.

- Build ChatInterface.tsx with scroll-to-bottom, keyboard avoidance
- Build MessageBubble.tsx with user/assistant variants
- Build TranscriptDisplay.tsx — load history from SQLite, append streaming messages
- Build StreamingText.tsx — render Claude's response token by token
- Build VoiceButton.tsx — connect to Member 2's useVoiceInput hook
- Build VoiceInputOverlay.tsx — recording state UI
- Build LoadingWave.tsx — waiting indicator
- Wire up Member 1's useConversation hook for the full flow

Task 2: Crisis UI (Priority 2)

- Build CrisisBar.tsx — appears on low/moderate risk, dismissible
- Build CrisisModal.tsx — appears mid-conversation on high risk, graceful transition
- Build SafetyCheckModal.tsx — full-screen, not dismissible, safety question + contacts
- Build crisis/index.tsx — full crisis pathway screen with static contact cards
- Wire up risk_level from Member 1's conversationService to trigger correct UI

Task 3: Toolkit (Priority 3)

- Build CopingCard.tsx and toolkit.tsx grid
- Build toolkit/[toolId].tsx detail screen
- Build toolkit/player.tsx audio player — connect to Member 2's useAudioPlayback
- Build AudioPlayer.tsx inline component

Task 4: Check-in Flow (Priority 4)

- Build EmotionWheelPicker.tsx, MoodFollowUp.tsx, CheckinSummary.tsx
- Build checkin/index.tsx screen
- Wire up to Member 4's moodRepository for saving check-ins
- Build MoodChart.tsx for profile screen

Task 5: Home & Profile (Priority 5)

- Build home.tsx — greeting, mood status, talk button, check-in prompt
- Build profile.tsx — nickname, language toggle, mood chart, about section
- Build LanguagePill.tsx and (tabs)/_layout.tsx

Dependencies

You Need From	What	When
---------------	------	------

Member 1	useConversation hook returning streaming text + risk_level	Before Task 1 integration
Member 2	useVoiceInput hook and useAudioPlayback hook	Before Task 1 integration
Member 4	Zustand stores (conversationStore, moodStore, userStore)	Before Task 1
Member 4	useMoodData hook for MoodChart	Before Task 4
Member 5	Color tokens, typography scale, emotion icons, i18n JSON files	Before Task 1 (can use placeholders initially)

Completion Checklist

- Conversation screen sends text, receives streaming response, displays it
- Voice button records, transcribes (via Khaya), sends to Claude
- TTS plays Claude's response in Twi
- CrisisBar appears on low/moderate risk
- CrisisModal triggers on high risk, cannot be dismissed without safety check
- Crisis screen shows contacts, safety question works
- Toolkit shows 2 tools, detail screen works, audio plays
- Check-in flow: emotion select → follow-up → save → summary
- MoodChart renders 7-day data on profile
- Home screen greeting works in both Twi and English
- Tab navigation works across Home, Toolkit, Profile
- Language toggle switches all UI text

Member 4 — Mobile Developer (Backend)

You own the data layer. If it saves, loads, or persists — it's yours.

Role Summary

You are responsible for all on-device data: the SQLite database, migrations, repositories (CRUD operations), Zustand state management, and the hooks that expose data to UI components. You also own the app's navigation skeleton (Expo Router layouts), environment configuration, and project setup. Your work is the foundation everyone else builds on — so your tasks need to be done early.

Files You Own

Storage Layer

File / Component	What You Build
src/services/storage/database.ts	Opens SQLite database via expo-sqlite. Runs migrations on first launch. Exports db instance.
src/services/storage/migrations/index.ts	Migration runner. Checks current schema version, applies pending migrations in order.
src/services/storage/migrations/001_initial.ts	Creates 4 tables: user_profile, conversation_sessions, messages, mood_checkins. Full schema.
src/services/storage/conversationRepository.ts	CRUD for conversations: createSession(), addMessage(), getSessionMessages(sessionId).
src/services/storage/moodRepository.ts	CRUD for mood_checkins: saveCheckin(), getCheckins(dateRange), getWeeklyTrend().
src/services/storage/userRepository.ts	Read/write for user_profile: getProfile(), updateProfile(), deleteProfile(). Stores: nickname, language code.

State Management

File / Component	What You Build
src/store/userStore.ts	Zustand store: nickname, language code, onboarding complete flag, consent flag. Persistent.
src/store/conversationStore.ts	Zustand store: current session ID, streaming state, last response text, current risk level.
src/store/moodStore.ts	Zustand store: today's check-in status, weekly mood summary (for home screen quick display).

Hooks

File / Component	What You Build
------------------	----------------

src/hooks/useConversation.ts	Manages a conversation session: loads history from SQLite, sends new messages to M
src/hooks/useMoodData.ts	Loads mood check-ins from SQLite for profile/chart screens. Provides weekly trend data
src/hooks/useLanguage.ts	Returns current language code and t(key) translation function. Listens to userStore for la

App Shell & Config

File / Component	What You Build
src/app/_layout.tsx	Root layout: sets up fonts (expo-font), initialises Zustand stores, hydrates from secure st
src/app/index.tsx	Entry: checks onboarding_complete flag, redirects accordingly.
app.json	Expo project config: name "Adwene Pa", permissions (microphone), bundle IDs, splash/
app.config.js	Dynamic config: reads ANTHROPIC_API_KEY and KHAYA_API_KEY from env.
babel.config.js	Babel config with path aliases (@/components, @/services, etc.).
tsconfig.json	TypeScript strict mode, path aliases.
package.json	All dependencies, scripts, Expo SDK version.

Constants & Types

File / Component	What You Build
src/constants/languages.ts	Array of 2 language objects: { code, displayName, khayaCode, nativeName }
src/constants/emotions.ts	Array of 10 emotion objects: { id, label: {tw, en}, iconPath, valence, arousal }
src/constants/crisisContacts.ts	4 contacts: Accra Psych Hospital, Mental Health Authority, Sandema Hospital, Emergen
src/constants/copingTools.ts	2 tool objects: { id, title: {tw, en}, description: {tw, en}, category, audioPath: {tw, en} }
src/constants/colors.ts	Design tokens from Member 5: primary, secondary, background, surface, text, error, cris
src/constants/typography.ts	Typography scale from Member 5: display, heading, body, caption sizes
src/types/*.ts	All TypeScript types: ConversationSession, Message, MoodCheckin, UserProfile, RiskL

Detailed Task Breakdown

Task 1: Project Scaffold (Do First)

- Initialize Expo project: `npx create-expo-app adwene-pa -t expo-template-blank-typescript`
- Install all dependencies (list below)
- Set up path aliases in `babel.config.js` and `tsconfig.json`
- Create the full directory structure from the project documentation
- Set up `.env.example` and `.gitignore`

- Create app.json with correct permissions (microphone)
- Push to GitHub, set up branch protection

Core Dependencies to Install

```
expo-sqlite expo-secure-store expo-av expo-speech expo-font
expo-haptics expo-router react-native-safe-area-context
react-native-screens react-native-svg zustand axios
@expo/vector-icons react-native-reanimated
```

Task 2: Database & Migrations

- Build database.ts — open/create SQLite database
- Build 001_initial.ts migration with all 4 tables:
 - user_profile: id, nickname, language_code, onboarding_done, consent_local, created_at
 - conversation_sessions: id (UUID), language_code, started_at, last_active_at, message_count, crisis_flagged
 - messages: id, session_id (FK), role, content, audio_uri, timestamp, risk_level
 - mood_checkins: id, emotion_id, emotion_label, follow_up_text, language_code, timestamp, session_id
- Build migration runner in index.ts
- Test: database creates, tables exist, basic insert/select works

Task 3: Repositories

- conversationRepository: createSession(), addMessage(), getSessionMessages(), listRecentSessions(), deleteSession()
- moodRepository: saveCheckin(), getCheckins(dateRange), getWeeklyTrend()
- userRepository: getProfile(), updateProfile(), deleteProfile()
- All repositories use the database.ts db instance
- Test each CRUD operation independently

Task 4: Zustand Stores

- userStore: nickname, language, onboarding flag, consent. Persist to expo-secure-store
- conversationStore: current sessionId, streaming state, last response, risk level
- moodStore: today's check-in status, weekly summary cache
- All stores use Zustand persist middleware with expo-secure-store adapter

Task 5: Hooks & App Shell

- useConversation: load history, coordinate with Member 1's conversationService, manage state
- useMoodData: query moodRepository, format for MoodChart component
- useLanguage: read language from userStore, provide t() translation function using i18n JSON
- Build _layout.tsx root: fonts, store hydration, navigation decision
- Build index.tsx: onboarding check and redirect

Task 6: Constants & Types

- Write all TypeScript type definitions
- Write all constants files using Member 5's design tokens and content
- Write crisisContacts.ts with verified phone numbers

Dependencies

You Need From	What	When
Member 5	Design tokens (colors, typography) and i18n JSON files (tw.js, en.js)	Before Task 4-6
Member 1	conversationService interface (what it calls, what it returns)	Before Task 5 (useConversation)
Nobody	You are the foundation — most of your work has no upstream dependency	

Completion Checklist

- Expo project initialised, dependencies installed, directory structure created
- GitHub repo created with branch protection
- SQLite database creates on first launch
- All 4 tables created by migration
- conversationRepository CRUD tested
- moodRepository CRUD tested
- userRepository CRUD tested
- userStore persists to secure storage and rehydrates on app restart
- conversationStore tracks session state
- moodStore tracks today's check-in
- useConversation hook coordinates service calls and state
- useMoodData hook returns chart-ready data
- useLanguage hook provides t() function with Twi and English
- Root layout loads fonts, hydrates stores, routes correctly
- All TypeScript types compiled without errors
- All constants files complete with correct data

Member 5 — UX / Content & Presenter

You own the feel. The design, the words, the Twi, and the pitch.

Role Summary

You are responsible for the visual design system, all Twi and English content (UI strings, coping tool scripts, safeguard statements), the onboarding flow screens, the i18n translation files, and the hackathon presentation materials. You are the bridge between the technical team and the end user. If it doesn't feel warm, Ghanaian, and inviting, it's your problem to fix. You also co-present with Member 1.

Files You Own

Design System

File / Component	What You Build
src/constants/colors.ts	Design tokens: primary (warm green #1B5E20), secondary (ochre #E65100), background
src/constants/typography.ts	Typography scale: display (28pt), heading (20pt), body (14pt), caption (12pt). Font family
assets/fonts/	Nunito font files: Regular, SemiBold, Bold. Download from Google Fonts.

Icons & Images

File / Component	What You Build
assets/icons/logo.svg	App logo: warm Kente-pattern feel, "A" letterform or Adinkra symbol. Keep simple.
assets/icons/tab-*.svg	3 tab icons: home, toolkit, profile. Consistent line weight. Warm, not clinical.
assets/icons/emotion-*.svg	10 emotion SVGs for the emotion wheel. Simple face/expression. Warm and approachable.
assets/images/icon.png	1024×1024 app icon. Kente-inspired background, white logo.
assets/images/splash.png	1284×2778 splash screen. Logo centred, deep green background, subtle Adinkra watermark.
assets/images/onboarding/	3 onboarding illustrations: welcome (diverse Ghanaians talking), voice (person speaking)

Content & Translations

File / Component	What You Build
src/i18n/tw.json	All UI string keys translated into Twi. Keys: home.greeting, home.how_are_you, tabs.toolkit
src/i18n/en.json	Same keys in English (the fallback).
src/i18n/index.ts	i18n loader: reads language code from userStore, loads correct JSON, exports lookup function

content/coping-tools/ grounding-54321.json	Grounding exercise content: title (tw/en), step-by-step body (tw/en), tags. Adapted for G
content/coping-tools/ breathing-478.json	Breathing exercise content: title (tw/en), step-by-step body (tw/en), tags.

Onboarding Screens

File / Component	What You Build
src/app/(onboarding)/_layout.tsx	Stack navigator. No back button on first screen. Progress indicator (step 1/4, 2/4, etc.).
src/app/(onboarding)/welcome.tsx	App name "Adwene Pa", tagline in English + Twi, "Get started" button. Background illust
src/app/(onboarding)/language-select.tsx	Two cards: Twi and English. Each plays a short audio sample when tapped. Clean, simp
src/app/(onboarding)/nickname.tsx	Optional nickname input. "What can we call you?" Skippable. Warm, no pressure.
src/app/(onboarding)/consent.tsx	Simplified: "I understand Adwene Pa is not a therapist." Single toggle for local mood dat

Presentation

File / Component	What You Build
Presentation deck (if applicable)	Slides or talking points for the team's hackathon presentation. Cover: problem stats, den
Demo script	Written script for the 90-second demo: exactly what to say at each timestamp, which but

Detailed Task Breakdown

Task 1: Design System (Do First)

- Define colour palette and write colors.ts — deliver to Member 4 immediately
- Define typography scale and write typography.ts
- Download Nunito fonts from Google Fonts, place in assets/fonts/

Colour palette reference:

Token	Hex	Usage
primary	#1B5E20	Headers, buttons, app identity
primaryLight	#4CAF50	Active states, selections
secondary	#E65100	Accents, CTAs
background	#FAFADF	Screen backgrounds
surface	#FFFFFF	Cards, modals

surfaceWarm	#FFF8E1	Assistant message bubbles
text	#212121	Primary text
textMuted	#616161	Secondary text, labels
userBubble	#E8F5E9	User message bubbles
error	#D32F2F	Error states
crisis	#B71C1C	Crisis UI elements
crisisBg	#FFE0E0	Crisis bar background

Task 2: Icons & Illustrations

- Design or source the 10 emotion SVGs — these appear in the emotion wheel, which is a key interaction
 - Happy, Sad, Anxious, Angry, Numb, Overwhelmed, Grateful, Lonely, Confused, Exhausted
 - Style: simple line art faces, warm, not clinical, not childish
- Design 3 tab icons (home, toolkit, profile) with consistent line weight
- Design app logo SVG
- Source or create 3 onboarding illustrations from Storyset.com or unDraw.co
- Generate app icon (1024×1024) and splash screen (1284×2778)

Task 3: Write All Twi & English UI Strings

This is a large content task. Start with the highest-priority screens:

- **Conversation screen strings:** "Kasa me ho" (Talk to me), "Meretie wo" (I'm listening), "Ka biribi" (Say something)
- **Crisis screen strings:** "Wo ho y■ wo?" (Are you safe?), "Aane" (Yes), "Dabi" (No), "Minnim" (Not sure)
- **Emotion labels:** All 10 emotions in Twi — anigye (happy), awer■ho■ (sad), ehu (anxious), abufuo (angry), etc.
- **Home screen:** "■te s■n" (How are you), greeting templates
- **Onboarding:** Welcome text, consent text, all button labels
- **Toolkit:** Tool names and descriptions in Twi
- **Safeguard statement:** The disclaimer in both Twi and English
- Write en.json first (fast), then tw.json (takes more care)

Task 4: Write Coping Tool Scripts

- Write English scripts for grounding (5-4-3-2-1) and breathing (4-7-8) — ~200 words each
- Adapt grounding for Ghana: "Name 5 things you can see in your compound..."
- Translate scripts to Twi (use Khaya translation API as first pass, then review manually)
- Write the JSON content files (grounding-54321.json, breathing-478.json)

- Deliver scripts to Member 2 for audio generation

Task 5: Validate Twi Content Across the App

- Review Member 1's twi.ts system prompt for natural Twi phrasing
- Test Claude's Twi responses — flag anything that sounds unnatural
- Verify all UI labels read naturally in Twi on screen (some translated phrases are too long)

Task 6: Build Onboarding Screens

- Build the 4 onboarding screens: welcome, language-select, nickname, consent
- Apply design system: colours, fonts, illustrations
- Implement progress indicator (step dots)
- Connect to Member 4's userStore for saving preferences

Task 7: Prepare Presentation

- Write the demo script with exact timestamps and actions
- Prepare talking points for the three mandatory questions
- Create any presentation materials (slides if applicable)
- Rehearse with Member 1 at least 3 times
- Prepare for judge Q&A — anticipate questions about cultural competence, your Twi expertise

Dependencies

You Need From	What	When
Member 4	userStore connected so onboarding saves preferences	Before Task 6
Member 1	twi.ts system prompt for Twi review	During Task 5
Member 2	Khaya translation API access for script translation	During Task 4
Nobody	Design system, icons, content — most of your work is independent	

Completion Checklist

- colors.ts and typography.ts delivered to Member 4
- Nunito fonts in assets/fonts/
- 10 emotion SVGs designed and in assets/icons/
- 3 tab icons designed
- App logo SVG designed
- App icon (1024×1024) and splash screen generated
- 3 onboarding illustrations sourced/created
- en.json complete with all UI string keys
- twi.json complete with all UI string keys — reviewed for natural Twi
- Grounding script written (English + Twi), JSON content file created

- Breathing script written (English + Twi), JSON content file created
- Scripts delivered to Member 2 for audio generation
- Member 1's twi.ts prompt reviewed for natural Twi
- Onboarding screens built and connected to stores
- Demo script written with exact timestamps
- Presentation rehearsed at least 3 times with Member 1

7. Shared Responsibilities & Handoff Points

Handoff Map

This diagram shows which team members depend on each other. Arrows point from provider to consumer.

```
Member 4 (Backend) ■■■■ Member 1 (AI Lead): conversationRepository API
Member 4 (Backend) ■■■■ Member 3 (Frontend): Zustand stores, hooks, types
Member 5 (UX/Content) ■■■■ Member 4 (Backend): design tokens, i18n JSON
Member 5 (UX/Content) ■■■■ Member 2 (NLP): coping tool scripts for audio
Member 5 (UX/Content) ■■■■ Member 1 (AI Lead): Twi validation
Member 1 (AI Lead) ■■■■ Member 3 (Frontend): conversationService, risk data
Member 2 (NLP) ■■■■ Member 3 (Frontend): useVoiceInput, useAudioPlayback hooks
```

Critical Handoff Deadlines

These are the points where one person's work blocks another. Miss these and the build stalls.

What	From	To	Deadline
Project scaffold + GitHub repo	Member 4	Everyone	Hour 1
Design tokens (colors.ts, typography.ts)	Member 5	Member 4	Hour 1
SQLite database + repositories	Member 4	Member 1	Hour 3
Zustand stores + types	Member 4	Member 3	Hour 3
i18n JSON files (tw.json, en.json)	Member 5	Member 4	Hour 4
base.ts system prompt (tested)	Member 1	Member 3	Hour 4
Khaya ASR/TTS working in Postman	Member 2	Member 2	Hour 2
useVoiceInput hook	Member 2	Member 3	Hour 5
Coping tool scripts (En + Tw)	Member 5	Member 2	Hour 5
conversationService streaming	Member 1	Member 3	Hour 5
Coping tool audio files (4)	Member 2	Member 3	Hour 7

Shared Responsibilities

- **Testing:** Everyone tests their own code before handoff. Don't hand off broken code.
- **Git discipline:** Everyone follows the branch strategy (Section 10). No direct pushes to main.
- **Communication:** When you finish a task that someone depends on, tell them immediately.
- **Scope questions:** If you're unsure whether to build something, ask Member 1. Default answer is "don't build it."

- **Demo preparation:** Everyone helps seed demo data in the last hour.

8. Hour-by-Hour Build Schedule

This assumes a 10-hour hackathon build window. Adjust proportionally for your actual time. The schedule front-loads risky and foundational work.

Hour 1 Setup

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Set up .env, review system prompts, install dependencies	Apply prompt strategy, key review, design system, scaffold project, initial dependencies	Review design, scaffold project, initial dependencies	Scaffold project, initial dependencies	Define personas, copyography, start emot

Hour 2 Foundation

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Write base.ts system prompt (first 400), test	Build first Chat.ts, test	Build SafeAreaWrapper, build bubbles, shared components, start en.json	Build Bubbles, shared components, start en.json	

Hour 3 Core Logic

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Refine base.ts, test with 100 inputs and API, test	Build inputs and API, test	Build MessageBubble, Building Vapp, States, Marked, write grounding script (E	Building Vapp, States, Marked, write grounding script (E	

Hour 4 Integration Prep

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Write twi.ts addendum, Build readerClient, test	Build readerClient, test	Build VoiceButtons, VoiceButtons, VoiceButtons, test	Build VoiceButtons, VoiceButtons, VoiceButtons, test	Translate registration to Twi, write bre

Hour 5 First Integration

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Build conversationService, test	Build service to DB, wire conversation screen	Build conversation screen	Build conversation screen	Build conversation screen

Hour 6 Voice Loop

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Test system prompts with 100 inputs, refine	Train ASR, refine conversation	Add voice input, build voice input, build voice input, test	Build voice input, build voice input, build voice input, test	Build voice input, build voice input, build voice input, test

Hour 7 Crisis & Tools

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Build riskScoring.ts, test	Consolidate prompting and build (As a Barty, Crisis) (Daher, crisis, and repository)	Build (As a Barty, Crisis) (Daher, crisis, and repository)	Build (As a Barty, Crisis) (Daher, crisis, and repository)	Build onboarding flow, username + consent

Hour 8 UI Polish

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)

Write CLINICAL_GUIDE into CLINICAL_GUIDE_SOP	Enhance voice chat SOP	Build 100% screens, and display data persisted	Build 100% screens, and display data persisted	Build 100% screens, and display data persisted
--	------------------------	--	--	--

Hour 9 Demo Prep

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Seed demo data (mood, Petres, checkspikes)	Build profile screen, Mood chat by integrating	Build profile screen, Mood chat by integrating	Build profile screen, Mood chat by integrating	Build profile screen, Mood chat by integrating

Hour 10 Rehearse

M1 (Lead/AI)	M2 (NLP)	M3 (Frontend)	M4 (Backend)	M5 (UX/Content)
Run full demo 3x, fix any issues	Standby for voice pipeline issues, responsive	Standby for voice pipeline issues, responsive	Standby for voice pipeline issues, responsive	Standby for voice pipeline issues, responsive

9. Integration Checkpoints

Stop and verify these milestones together as a team. If a checkpoint fails, fix it before moving on.

Checkpoint	When	What Must Work	Who Tests
CP1: Foundation	End of Hour 2	Expo app runs on device/emulator. SQLite database created. Fonts load.	M4 + M3
CP2: Text Conversation	End of Hour 5	Type a message → Claude responds → response displays on chat → saves to DB	M1 + M3 + M4
CP3: Voice Loop	End of Hour 6	Speak Twi → Khaya transcribes → Claude responds → Khaya speaks response	M1 + M2 + M3
CP4: Crisis Path	End of Hour 7	Send a crisis message → risk_level=high → CrisisModal appears → contacts shown	M1 + M2
CP5: Full Demo	End of Hour 9	Complete 90-second demo flow runs without errors	Everyone

If CP3 (Voice Loop) fails because Khaya is down, fall back to text-only demo. This is still a strong submission — the system prompts, crisis detection, and cultural competence are the real value.

10. Git Workflow & Branch Strategy

Branch Naming

Branch	Owner	Purpose
main	Protected	Only merged into after checkpoint passes
dev	All	Integration branch — merge features here first
feat/system-prompts	M1	System prompts + Claude integration
feat/khaya-voice	M2	Khaya ASR/TTS + audio services
feat/conversation-ui	M3	Conversation screen + crisis UI
feat/data-layer	M4	SQLite + stores + hooks
feat/design-content	M5	Design system + i18n + onboarding

Workflow Rules

- Work on your own feature branch
- When a task is complete and tested, merge to dev
- If dev breaks after a merge, the person who merged last fixes it
- Only merge dev → main after an integration checkpoint passes
- Commit messages: "[M1] Add base system prompt" or "[M3] Build conversation screen"
- Never force push to dev or main

11. Environment Setup (Everyone)

Prerequisites

- Node.js 18+ installed
- npm or yarn installed
- Expo Go app installed on your phone (for testing)
- Git configured with your GitHub account
- VS Code (recommended) with ESLint and Prettier extensions

First-Time Setup

```
git clone [repo-url]
cd adwene-pa
npm install
cp .env.example .env
# Add your API keys to .env:
# ANTHROPIC_API_KEY=sk-ant-...
# KHAYA_API_KEY=...
npx expo start
```

.env File

```
ANTHROPIC_API_KEY=sk-ant-xxxx
KHAYA_API_KEY=xxxx
```

Member 1 manages the Anthropic API key. Member 2 manages the Khaya API key. Share keys via secure channel (not in git, not in group chat).

12. Emergency Protocols

Things will go wrong during the hackathon. Here's how to handle the most likely failures.

Emergency	Response	Owner
Khaya API is down	Switch to text-only mode. Remove voice button from demo. Frame M2 as "AI text assistant" as fallback.	M2
Claude API credits exhausted	Use mock responses for UI testing. Cache working conversations for demo.	M1
Expo build fails on device	Test on emulator instead. If emulator also fails, screen-record from a working device.	M1
Database corruption	Delete app data, reinstall. Migrations will recreate everything. Reset demo data.	M4
Team member drops out	M1 absorbs AI tasks. M3 absorbs any frontend. M4 absorbs any backend. M5's content can be simplified.	M1
Demo crashes during judging	Play pre-recorded backup video (M2 records in Hour 9). Switch immediately to show you what this is.	M2
Twili translation sounds weird	Switch back to English for the demo. Mention Twili as roadmap: "We've made Twili support and are refining the	M3

Remember: The system prompts are the product. The voice loop is the wow factor. The crisis detection is the ethical requirement. Everything else is polish. Build in that order, and you'll have a submission that scores high on all four judging criteria.

Adwene Pa — because you can't heal in a language that doesn't feel like home.